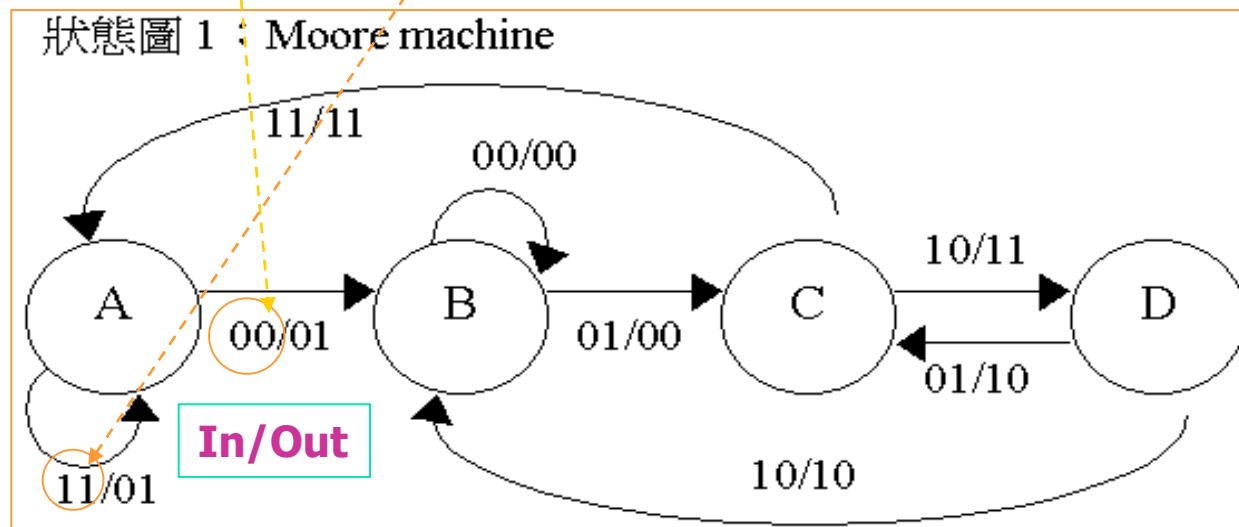


Verilog 硬體描述語言 (I)

- 有限狀態機器
- 有限狀態機的設計實例

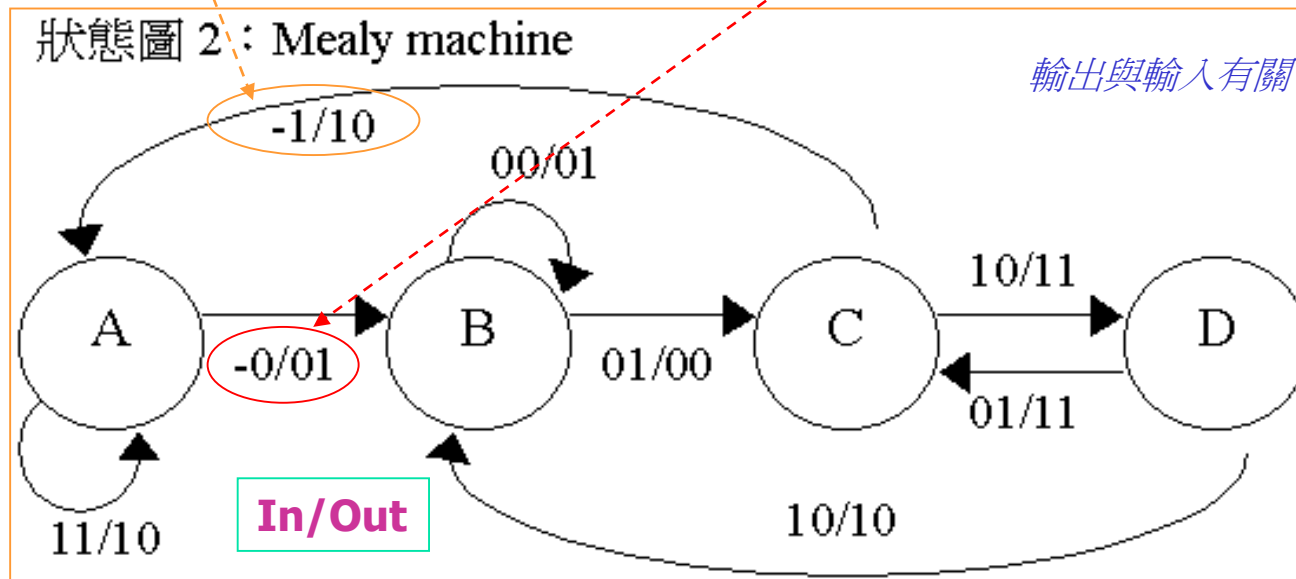
Moore Machine

- ❑ 「有限狀態機器」(Finite State Machine, FSMs)：其係循序電路中變化性最多的一種循序邏輯電路，通常用在比較特殊要求的控制電路中。暫存器、移位暫存器、計數器等均可以看成是一種功能固定的有限狀態機器。
- ❑ 有限狀態機器 依其輸入的訊號、輸出的訊號及狀態之間的關係，可細分為二大類：Moore machine 及 Mealy (有斑點的) Machine。
- ❑ Moore machine：這種有限狀態機器，其輸出的訊號只與目前的狀態有關，而與輸入的訊號無關。以「Moore machine」的狀態 A 來說，不管輸入訊號給 00 或是 11，其輸出訊號皆會是 01。



Mealy Machine

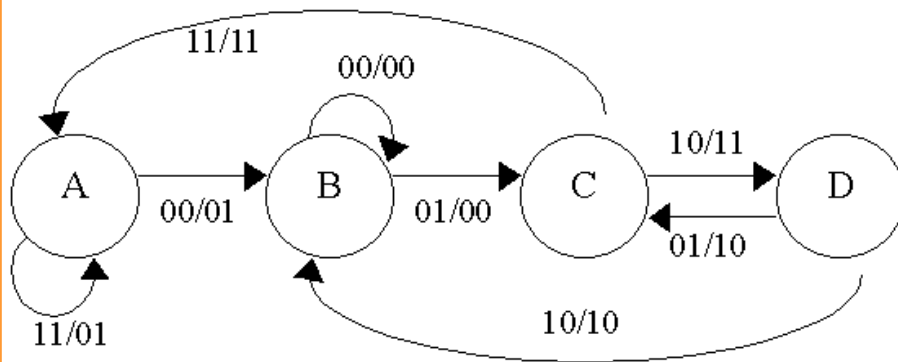
- ❑ Mealy(撒粉的,有斑點的) Machine：這種有限狀態機器其輸出的訊號必需考慮目前的狀態及輸入的訊號。
- ❑ Mealy Machine 雖然可以用較少的狀態變數來實現一個有限狀態機器，但其輸出訊號會受到輸入訊號的干擾，因而容易在輸出訊號上產生「突波」現象。
- ❑ 如果延長時脈的週期，讓電路有較長的時間處於穩態的狀況，相對的發生突波的時間就顯得小多了。



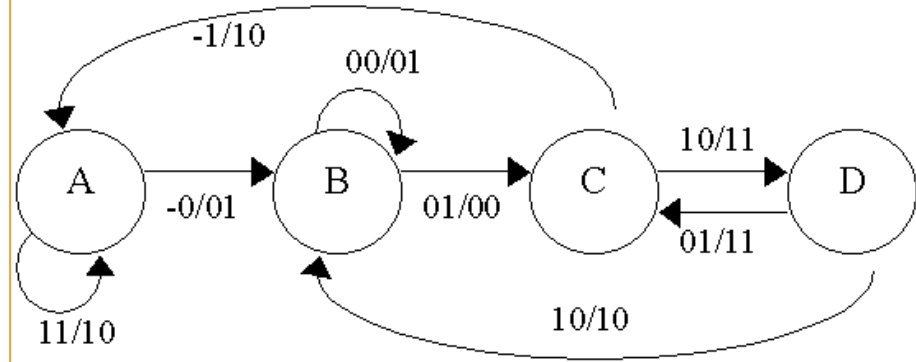
State Transition Graph (STG)

- 在狀態圖中，每一個頂點（圓圈）代表循序電路中的一個狀態，頂點（圓圈）內的符號代表該狀態的名稱，每一個有方向性的分支 (Directed Branch) 代表循序電路的一個狀態轉移，有方向性的分支旁的標記代表該電路轉移所對應的輸入訊號的變化與輸出訊號的變化。
- 例如：在上一小節所討論的有限狀態機的種類中，「狀態圖1：Moore machine」及「狀態圖2：Mealy machine」均是使用狀態圖的方式來表示所需要的有限狀態機器。

狀態圖 1：Moore machine



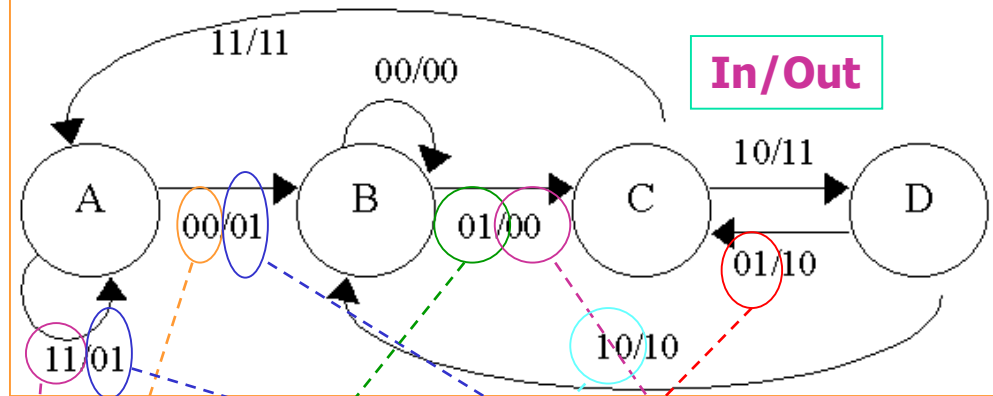
狀態圖 2：Mealy machine



State Transition Table (STT)

- 在狀態表中，每一列相當於是在狀態圖中所包含的一個狀態轉移動作。
- 「狀態表 1：Moore machine」是使用狀態表的表示方式。
- 從該表格中，可以很明顯地看出，不管目前狀態為何，它的輸出訊號一定相同。這點說明了：Moore machine 其輸出的訊號只與目前的狀態有關而與輸入的訊號無關。

狀態圖 1：Moore machine



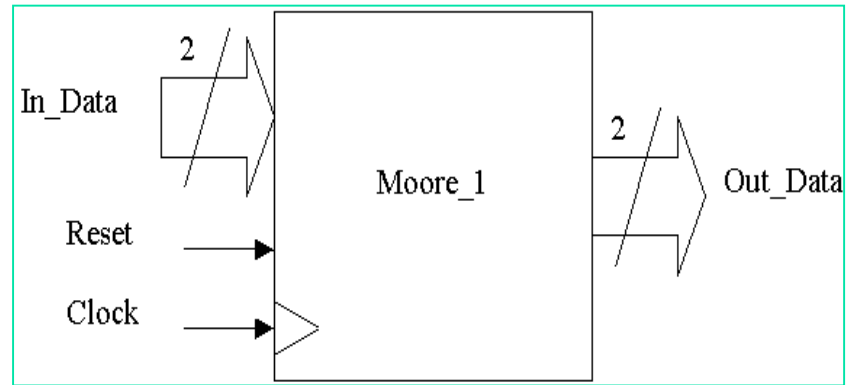
狀態表 1：Moore machine

輸入訊號	目前狀態	下一狀態	輸出訊號
11	A	A	01
00	A	B	01
00	B	B	00
01	B	C	00
10	C	D	11
11	C	A	11
10	D	B	10
01	D	C	10

STT of Moore Machine (1)

//Moore_1.V：狀態表1：Moore machine的 Verilog 電路描述
module Moore_1 (Clock, Reset, In_Data, Out_Data);

input Clock, Reset;
input [1:0] In_Data;
output [1:0] Out_Data;
reg [1:0] Out_Data;
reg [1:0] state;



parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;

always @(posedge Clock or posedge Reset)
begin

if (Reset) // 電路重置

begin

state = A; // 初始狀態設為 A

end

完全指定狀態

STT of Moore Machine (2)

```
else  
begin
```

```
  case (state)
```

```
    A: // 目前狀態為 A
```

```
    begin
```

```
      if (In_Data = 2'b11)
```

```
      begin
```

```
        Out_Data = 2'b01;
```

```
        state = A; // 下個狀態為 A
```

```
      end
```

```
      if (In_Data = 2'b00)
```

```
      begin
```

```
        Out_Data = 2'b01;
```

```
        state = B; // 下個狀態為 B
```

```
      end
```

```
    end
```

狀態表 1 : Moore machine

輸入訊號	目前狀態	下一狀態	輸出訊號
11	A	A	01
00	A	B	01
00	B	B	00
01	B	C	00
10	C	D	11
11	C	A	11
10	D	B	10
01	D	C	10

STT of Moore Machine (3)

狀態表 1 : Moore machine

輸入訊號	目前狀態	下一狀態	輸出訊號
11	A	A	01
00	A	B	01
00	B	B	00
01	B	C	00
10	C	D	11
11	C	A	11
10	D	B	10
01	D	C	10

B: // 目前狀態為 B

```
begin
```

```
  if (In_Data = 2'b00)
```

```
  begin
```

```
    Out_Data = 2'b00;
```

```
    state = B; // 下個狀態為 B
```

```
  end
```

```
  if (In_Data = 2'b01)
```

```
  begin
```

```
    Out_Data = 2'b00;
```

```
    state = C; // 下個狀態為 C
```

```
  end
```

```
end
```


STT of Moore Machine (4)

狀態表 1：Moore machine

輸入訊號	目前狀態	下一狀態	輸出訊號
11	A	A	01
00	A	B	01
00	B	B	00
01	B	C	00
10	C	D	11
11	C	A	11
10	D	B	10
01	D	C	10

C: // 目前狀態為 C

```
begin
```

```
if (In_Data = 2'b10)
```

```
begin
```

```
    Out_Data = 2'b11;
```

```
    state = D; // 下個狀態為 D
```

```
end
```

```
if (In_Data = 2'b11)
```

```
begin
```

```
    Out_Data = 2'b11;
```

```
    state = A; // 下個狀態為 A
```

```
end
```

```
end
```

STT of Moore Machine (5)

狀態表 1 : Moore machine

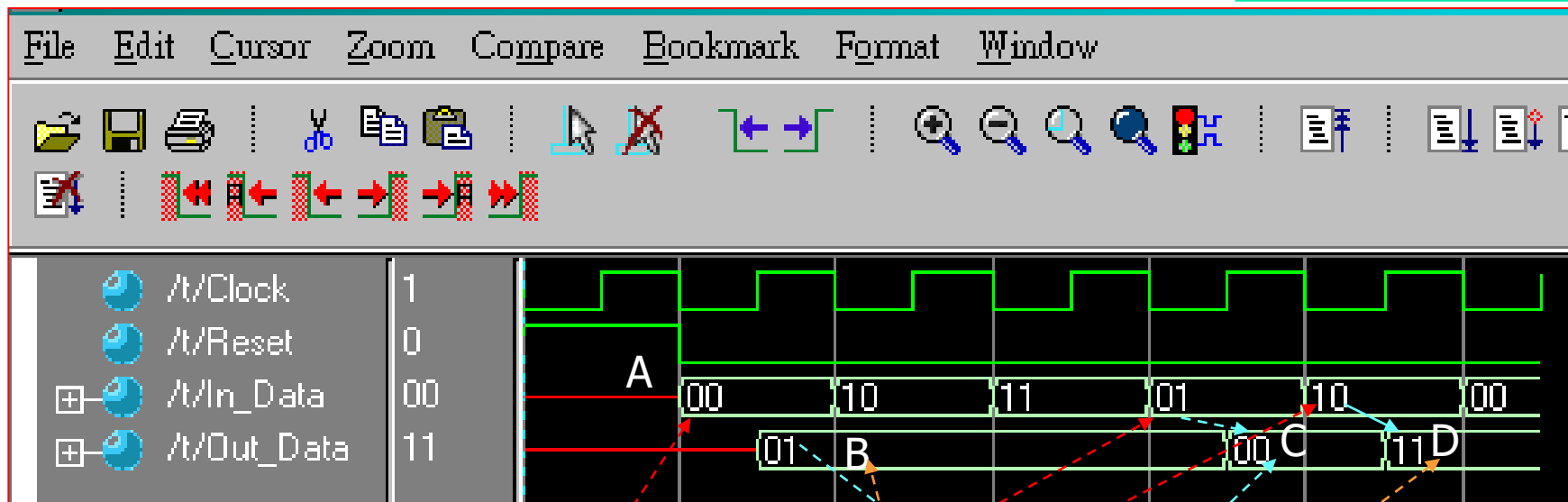
輸入訊號	目前狀態	下一狀態	輸出訊號
11	A	A	01
00	A	B	01
00	B	B	00
01	B	C	00
10	C	D	11
11	C	A	11
10	D	B	10
01	D	C	10

```
D: // 目前狀態為 D
begin
  if (In_Data = 2'b10)
  begin
    Out_Data = 2'b10;
    state = B; // 下個狀態為 B
  end
  if (In_Data = 2'b01)
  begin
    Out_Data = 2'b10;
    state = C; // 下個狀態為 C
  end
end
endcase
end
end
endmodule
```

此種設計不好，
當輸入為00、11時為unknown

Simulation Waveform

4_27_moore_1



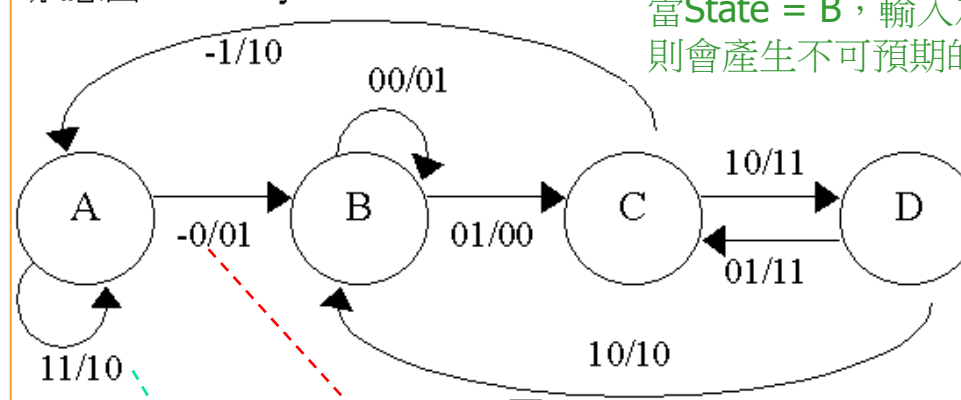
狀態表 1 : Moore machine

輸入訊號	目前狀態	下一狀態	輸出訊號
11	A	A	01
00	A	B	01
00	B	B	00
01	B	C	00
10	C	D	11
11	C	A	11
10	D	B	10
01	D	C	10

STT of Mealy machine (1)

- 從該狀態表中，
可以很明顯地
看出，Mealy
machine 的輸
出訊號必需考
慮目前的狀態
及輸入的訊號。

狀態圖 2：Mealy machine



當State = B，輸入為10 / 11 時，
則會產生不可預期的結果

狀態表 2：Mealy machine

輸入訊號	目前狀態	下一狀態	輸出訊號
-0 0	A	B	01
11 1	A	A	10
00 0	B	B	01
01 1	B	C	00
10 0	C	D	11
-1 1	C	A	10
10 0	D	B	10
01 1	D	C	11

STT of Mealy machine (2)

//Mealy_1.V : 狀態表2 : Mealy machine的Verilog電路描述

```
module Mealy_1 (Clock, Reset, In_Data, Out_Data);
```

```
input    Clock, Reset;
```

```
input    [1:0] In_Data;
```

```
output   [1:0] Out_Data;
```

```
reg      [1:0] Out_Data;
```

```
reg      [1:0] state;
```

狀態表 2 : Mealy machine

輸入訊號	目前狀態	下一狀態	輸出訊號
-0	A	B	01
11	A	A	10
00	B	B	01
01	B	C	00
10	C	D	11
-1	C	A	10
10	D	B	10
01	D	C	11

```
parameter A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;
```

```
always @(posedge Clock or posedge Reset)
```

```
begin
```

```
  if (Reset) // 電路重置
```

```
  begin
```

```
    state = A; // 初始狀態設為 A
```

```
    Out_Data = 2'b00;
```

```
  end
```

加入此行，
目的讓輸出值為已知

STT of Mealy machine (3)

```
else  
begin
```

```
  case (state)
```

```
    A: // 目前狀態為 A
```

```
      begin
```

```
        if ((In_Data == 2'b00)|| (In_Data == 2'b10))
```

```
          begin
```

```
            Out_Data = 2'b01;
```

```
            state = B; // 下個狀態為 B
```

```
          end
```

```
          if (In_Data == 2'b11)
```

```
            begin
```

```
              Out_Data = 2'b10;
```

```
              state = A; // 下個狀態為 A
```

```
            end
```

```
          end
```

X0 = -0

狀態表 2：Mealy machine

輸入訊號	目前狀態	下一狀態	輸出訊號
-0	A	B	01
11	A	A	10
00	B	B	01
01	B	C	00
10	C	D	11
-1	C	A	10
10	D	B	10
01	D	C	11

STT of Mealy machine (4)

B: // 目前狀態為 B

begin

if (In_Data == 2'b00)

begin

Out_Data = 2'b01;

state = B; // 下個狀態為 B

end

if (In_Data == 2'b01)

begin

Out_Data = 2'b00;

state = C; // 下個狀態為 C

end

end

狀態表 2 : Mealy machine

輸入訊號	目前狀態	下一狀態	輸出訊號
-0	A	B	01
11	A	A	10
00	B	B	01
01	B	C	00
10	C	D	11
-1	C	A	10
10	D	B	10
01	D	C	11

STT of Mealy machine (5)

C: // 目前狀態為 C

begin

if (In_Data == 2'b10) ———▶

begin

Out_Data = 2'b11;

state = D; // 下個狀態為 D

end

if ((In_Data == 2'b01) || (In_Data == 2'b11))

begin

Out_Data = 2'b10;

state = A; // 下個狀態為 A

end

end

狀態表 2：Mealy machine

輸入訊號	目前狀態	下一狀態	輸出訊號
00	A	B	01
11	A	A	10
00	B	B	01
01	B	C	00
10	C	D	11
01	C	A	10
10	D	B	10
01	D	C	11

STT of Mealy machine (6)

D: // 目前狀態為 D

```
begin
```

```
  if (In_Data == 2'b10)
```

```
  begin
```

```
    Out_Data = 2'b10;
```

```
    state = B; // 下個狀態為 B
```

```
  end
```

```
  if (In_Data == 2'b01)
```

```
  begin
```

```
    Out_Data = 2'b11;
```

```
    state = C; // 下個狀態為 C
```

```
  end
```

```
end
```

```
endcase
```

```
end
```

```
end
```

```
endmodule
```

狀態表 2：Mealy machine

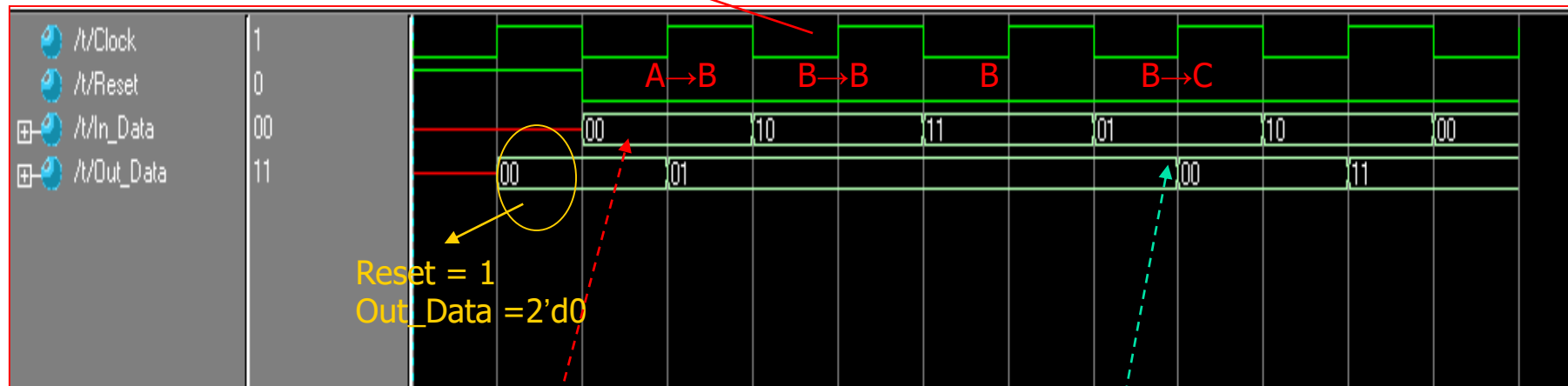
輸入訊號	目前狀態	下一狀態	輸出訊號
-0	A	B	01
11	A	A	10
00	B	B	01
01	B	C	00
10	C	D	11
-1	C	A	10
10	D	B	10
01	D	C	11

此種設計不好，
當輸入為00、11時為unknown

Simulation Waveform(2)

此種方法不好會有未知情形

4_28_MEALY_1



狀態表 2：Mealy machine

輸入訊號	目前狀態	下一狀態	輸出訊號
-0	A	B	01
11	A	A	10
00	B	B	01
01	B	C	00
10	C	D	11
-1	C	A	10
10	D	B	10
01	D	C	11

FSM Encoding (1)

❑ 「二進制碼」(Binary Code) 編碼方式

二進制碼編碼方式是一種最小長度 (minimal length) 的編碼方式，所以用於編碼所需的正反器數目會是最少的。

❑ 「格雷碼」(Gray Code) 編碼方式

➤ 格雷碼編碼方式也是一種最小長度 (minimal length) 的編碼方式，所以用於編碼所需的正反器數目也是最少的。

0	00
1	01
2	11
3	10

➤ 在格雷碼編碼方式中，二個相鄰的狀態被選擇的編碼只有一個位元是不同的；也就是用在電路中，只有一個正反器的值會發生變化，因此如果先分析電路狀態的轉移關係再配合使用這種編碼方式，可以讓電路的消耗功率較低。

❑ 「One-Hot 碼」編碼方式

One-Hot 碼編碼方式是一種非最小長度 (Non-minimal length) 的編碼方式，所以用於編碼所需的正反器數目較其他採用最小長度 (minimal length) 的編碼方式編碼所需的正反器數來得多，而且所需的正反器數目就等於狀態機的狀態數目。

FSM Encoding (2)

- ❑ 在 K-Hot 碼中，假設「 r 為狀態變數的正反器值有1的數目」、「 n 為用於編碼所需的正反器數目(位元數)」，那麼總共可用於編碼的數目有 $C(n, r) = n! / (r! * (n-r)!)$ 個。
- ❑ **範例1**：假設狀態變數的”正反器值有1”的數目為2個，則用於編碼所需的正反器數目為4個位元，那麼總共可用於編碼的數目有 $C(4, 2) = 4! / [(2!) * (2!)] = 6$ 個。
 - 包括那6個呢？
1. 0011、2. 0101、3. 1001、4. 0110、5. 1010、6. 1100。
- ❑ **範例2**：假設狀態變數的”正反器值有1”的數目為2個，而用於編碼所需的正反器數目為5個，那麼總共可用於編碼的數目有 $C(5, 2) = 5! / (2!) * (3!) = 10$ 個。
 - 包括那10個呢？
1. 00011、2. 00101、3. 01001、4. 10001、5. 00110、6. 01010、7. 10010、8. 01100、9. 10100、10. 11000。

FSM Encoding (3)

//Sum3Data.V 的 Verilog 電路描述

// 每當有一個 Clock 訊號來時，就將輸入的資料值加至計數值中，

// 連加三個輸入的資料後，電路的狀態改為初始狀態，

// 也就是當下一個 Clock 訊號來時計數值改為輸入的資料值 + 計數值。

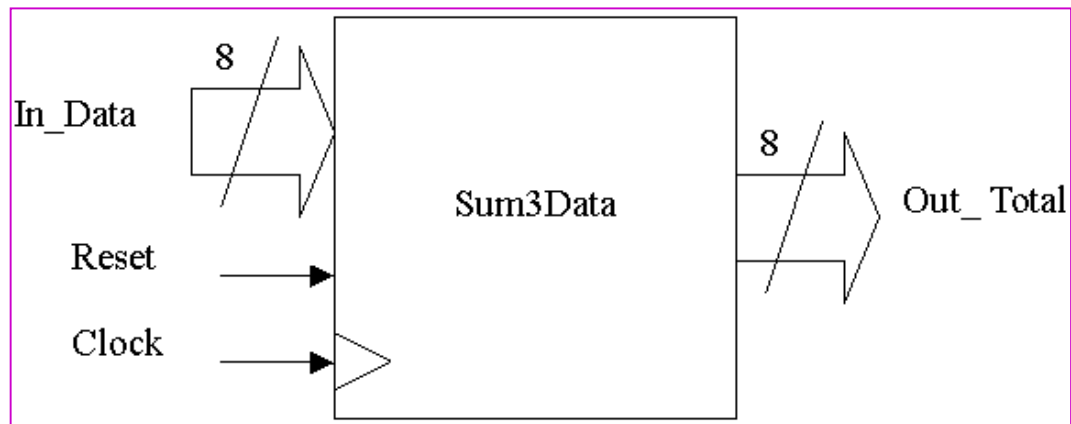
```
module Sum_3_Continuous_Input_Data(Clock, Reset, In_Data, Out_Total);
```

```
input    Clock, Reset;
```

```
input    [7:0] In_Data;
```

```
output   [7:0] Out_Total;
```

```
reg      [7:0] Out_Total;
```



```
reg [1:0] state;
```

// 因為採用二進制碼編碼方式，3個狀態需2個正反器 (也就是 reg [1:0])

// 如果改用One-Hot 碼編碼方式 (001, 010, 100)，則需3個正反器 (reg [2:0])

// 如果選用 (3, 2)-碼編碼方式 (011,101,110)，同樣需要3個正反器 (reg [2:0])

```
parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10;
```

FSM Encoding (4)

```
always @(posedge Clock)
```

```
begin
```

```
  if (Reset) // 電路重置
```

```
  begin
```

```
    state = s0; // 初始狀態為 s0
```

```
  end
```

```
  else
```

```
  begin
```

```
    case (state)
```

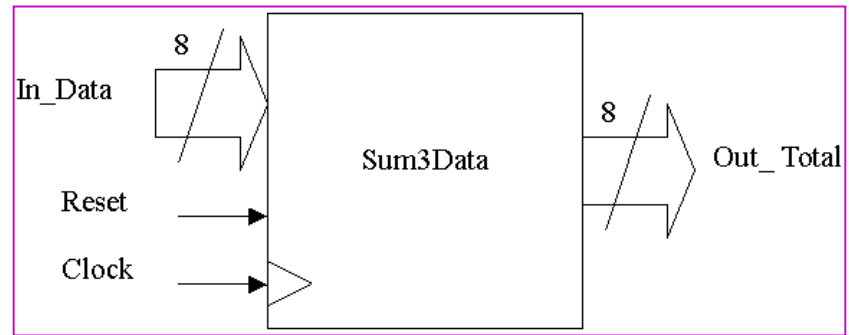
```
      s0: // 目前狀態為 s0
```

```
      begin
```

```
        Out_Total = In_Data;
```

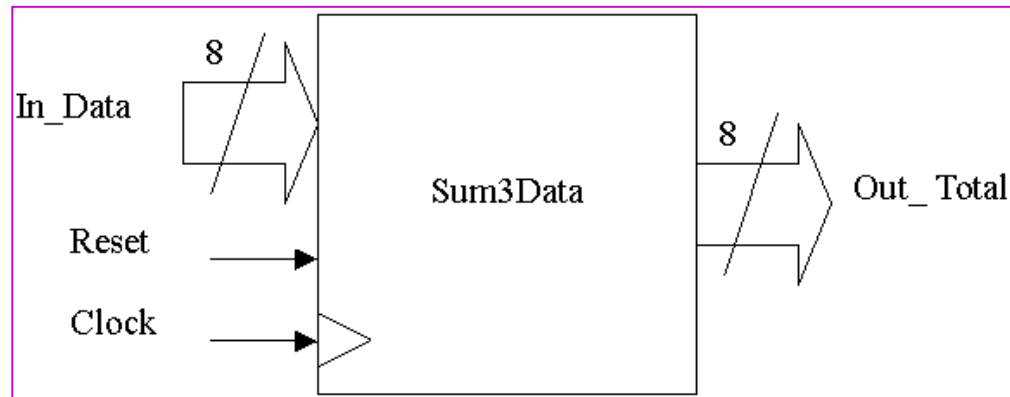
```
        state = s1; // 下個狀態為 s1 (加一次)
```

```
      end
```



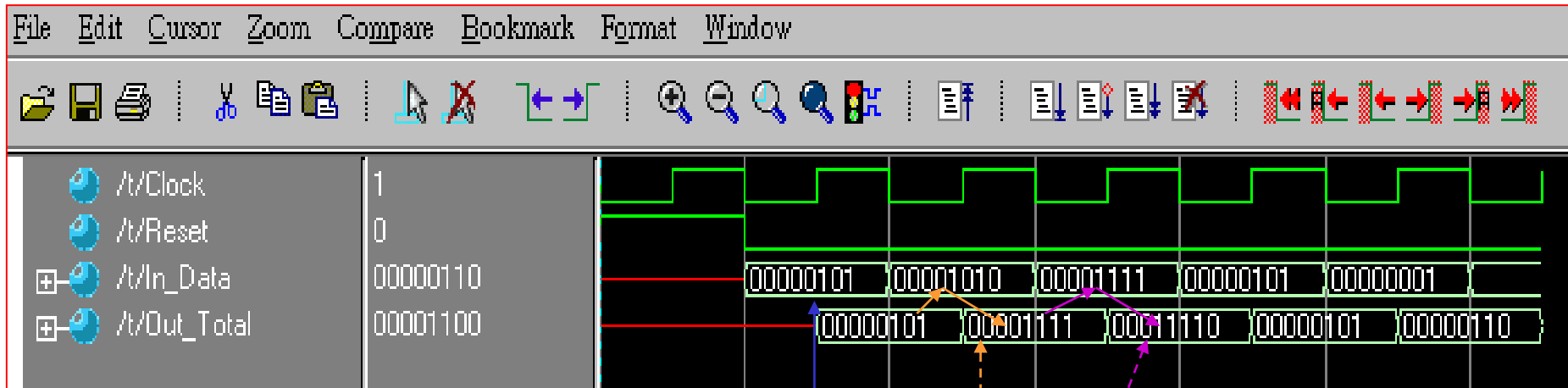
FSM Encoding (5)

```
s1: // 目前狀態為 s1
begin
    Out_Total = Out_Total + In_Data;
    state = s2; // 下個狀態為 s2(加二次)
end
default: // 其他狀態
begin
    Out_Total = Out_Total + In_Data;
    state = s0; // 下個狀態為 s0 (加三次，回至s0)
end
endcase
end
endmodule
```

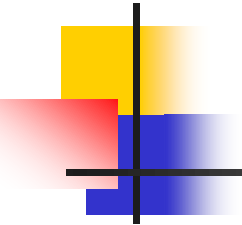


Simulation Waveform

4_31_SUM3DATA

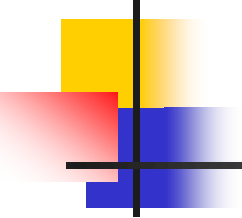


```
case (state)
s0: // 目前狀態為 s0
begin
    Out_Total = In_Data;
    state = s1; // 下個狀態為 s1 (加一次)
end
s1: // 目前狀態為 s1
begin
    Out_Total = Out_Total + In_Data;
    state = s2; // 下個狀態為 s2 (加二次)
end
default: // 其他狀態
begin
    Out_Total = Out_Total + In_Data;
    state = s0; // 下個狀態為 s0 (加三次, 回至s0)
end
```

Verilog 硬體描述語言 (I)

- 有限狀態機器
- 有限狀態機的設計實例



範例練習

Moore machine(10-001)

奇同位核對電路之

Moore狀態機(二進制編碼法)

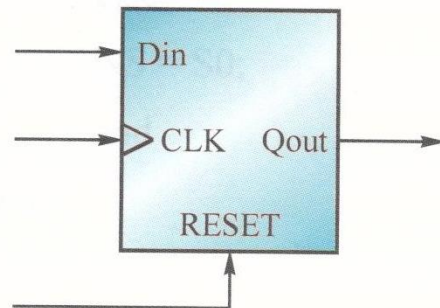
Moore Machine with Binary Code

利用二進制編碼法來設計奇同位核對器，只要輸入信號Din中有奇數個(1)時，則輸出 $Q_{out}=1$ ，否則輸出 $Q_{out}=0$ 。

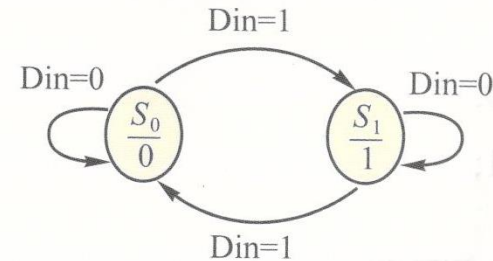
動作範例：

Initial status
= S_0

Din	-	0	1	1	1	0	1
Q_{out}	0	0	1	0	1	1	0
Status	S_0	S_0	S_1	S_0	S_1	S_1	S_0



(a) 方塊圖



(b) 狀態圖

目前狀態	下一個狀態		輸出 Q_{out}
	Din=0	Din=1	
$S_0=0$	S_0	S_1	0
$S_1=1$	S_1	S_0	1

(c) 狀態表

```
//Moore machine with Binary encoded stste machine  
module MOOR_BIN2(CLK, Din,RESET, Qout);
```

```
    input CLK, RESET;
```

```
    input Din;
```

```
    output Qout;
```

```
    reg Qout;
```

```
    parameter //Declare for all states
```

```
        S0 = 1'b1,
```

```
        S1 = 1'b0;
```

```
    reg CS; // Current state
```

```
    reg NS; // Next state
```

```
always@(posedge CLK or posedge RESET)
```

```
    begin
```

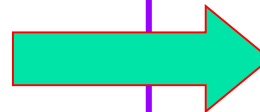
```
        if(RESET == 1'b1)
```

```
            CS = S0;
```

```
        else
```

```
            CS = NS;
```

```
    end
```



```
    always@(CS or Din)
```

```
        begin
```

```
            case(CS)
```

```
                S0: begin
```

```
                    Qout = 1'b0;
```

```
                    if(Din == 1'b0)
```

```
                        NS = S0;
```

```
                    else
```

```
                        NS = S1;
```

```
                end
```

```
                S1: begin
```

```
                    Qout = 1'b1;
```

```
                    if(Din == 1'b0)
```

```
                        NS = S1;
```

```
                    else
```

```
                        NS = S0;
```

```
                end
```

```
            endcase
```

```
        end
```

```
    endmodule
```

```

module MOOR_BIN2_tb();
    reg CLK, RESET;
    reg Din;
    wire Qout;
    MOOR_BIN2
    MOOR_BIN2(.CLK(CLK), .Din(Din), .RESET(RESET), .Qout(Qout));

    initial
        begin
            #0 RESET = 1'b1;
            CLK = 1'b0;
            #20 RESET = 1'b0;
            #5 Din = 0;
            #20 Din = 1;
            #20 Din = 0;
            #20 Din = 1;
        end

    always
        #10 CLK = ~CLK;

    initial
        #150 $stop;

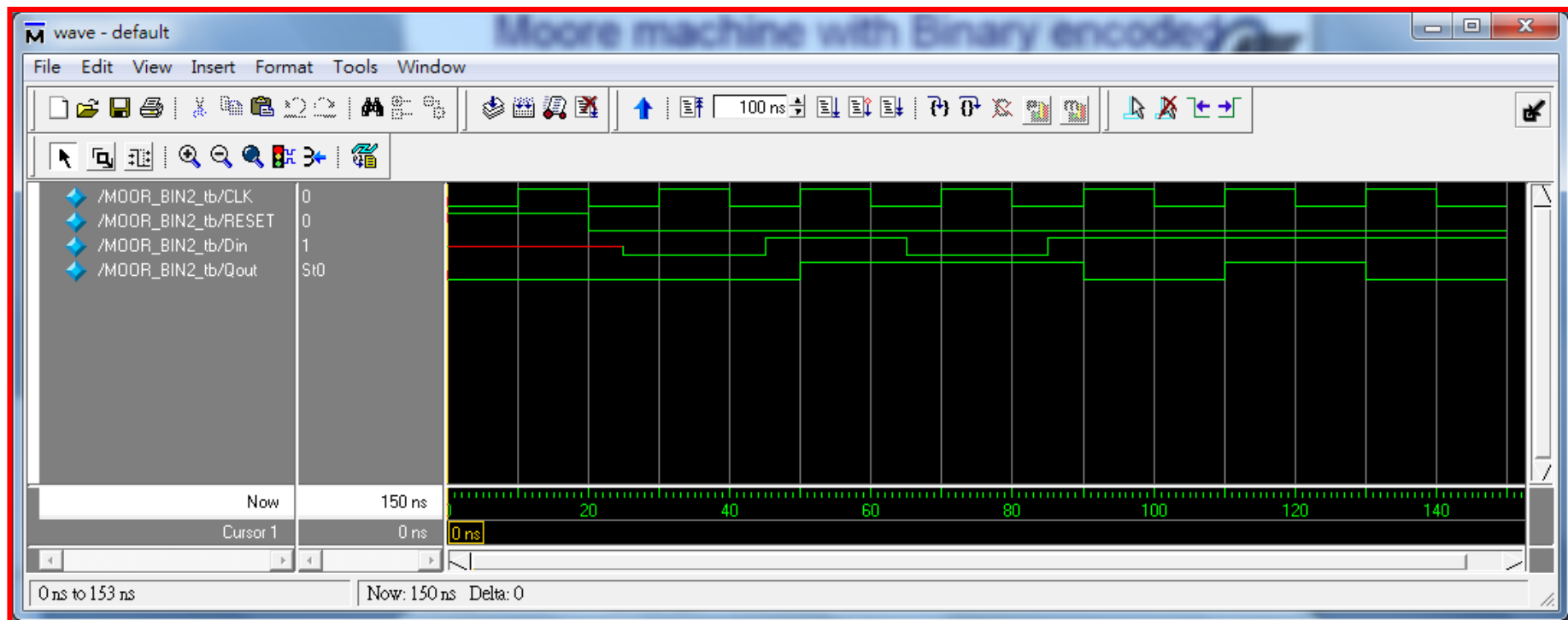
endmodule

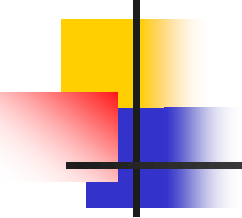
```

Testbench

Simulation Waveform

Code:10_001





範例練習

Moore machine(10-002)

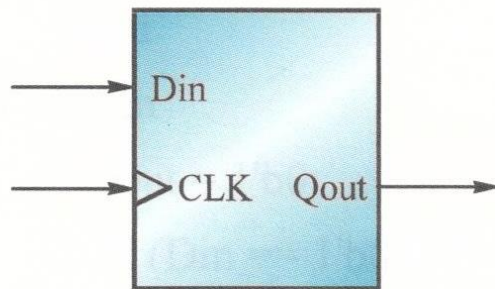
連續輸入二個(0)後的

奇同位核對電路(單點熱編碼法)

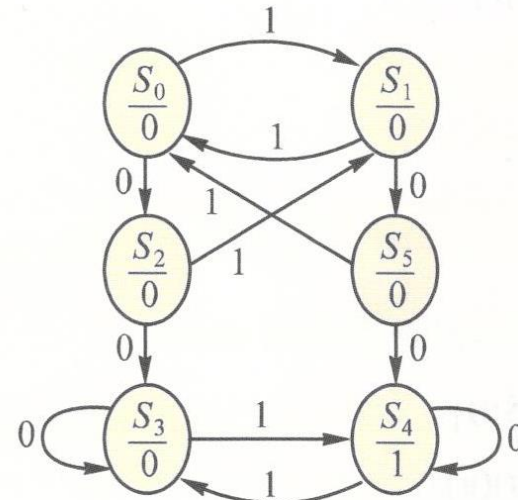
Moore Machine with One-hot Code

- ❑ 利用單點熱編碼方式來設計”連續輸入出現二個(0)以後，再開始輸出奇同位元核對器。
- ❑ 奇同位核對器：只要輸入信號 D_{in} 中有奇數個(1)時，則輸出 $Q_{out}=1$ ，否則輸出 $Q_{out}=0$ 。
- ❑ 動作範例：
- ❑ **Initial status**
= S_0

Din	-	1	0	0	0	0	1	1	1
Q_{out}	0	0	0	1	1	1	0	1	0
Status	S_0	S_1	S_5	S_4	S_4	S_4	S_3	S_4	S_3



(a) 方塊圖



(b) 狀態圖


```
//Moore machine with One-hot encoded state machine
```

```
module MOOR_ONEH2(CLK, Din, RESET, Qout);
```

```
    input CLK, RESET;
```

```
    input Din;
```

```
    output Qout;
```

```
    reg Qout;
```

```
    parameter[5:0]
```

```
        S0 = 6'b000001,
```

```
        S1 = 6'b000010,
```

```
        S2 = 6'b000100,
```

```
        S3 = 6'b001000,
```

```
        S4 = 6'b010000,
```

```
        S5 = 6'b100000;
```

```
    reg [5:0]CS;
```

```
    reg [5:0]NS;
```

```
    always@(posedge CLK or
             posedge RESET)
```

```
        begin
```

```
            if(RESET == 1'b1)
```

```
                CS = S0;
```

```
            else
```

```
                CS = NS;
```

```
        end
```

```
    always@(CS or Din)
```

```
        begin
```

```
            case(CS)
```

```
                S0:
```

```
                    begin
```

```
                        Qout = 1'b0;
```

```
                        if(Din == 1'b0)
```

```
                            NS = S2;
```

```
                        else
```

```
                            NS = S1;
```

```
                    end
```

```
                S1:
```

```
                    begin
```

```
                        Qout = 1'b0;
```

```
                        if(Din == 1'b0)
```

```
                            NS = S5;
```

```
                        else
```

```
                            NS = S0;
```

```
                    end
```

```
                S2:
```

```
                    begin
```

```
                        Qout = 1'b0;
```

```
                        if(Din == 1'b0)
```

```
                            NS = S3;
```

```
                        else
```

```
                            NS = S1;
```

```
                    end
```

```
                S3:
```

```
                    begin
```

```
                        Qout = 1'b0;
```

```
                        if(Din == 1'b0)
```

```
                            NS = S3;
```

```
                        else
```

```
                            NS = S4;
```

```
                    end
```

```
                S4:
```

```
                    begin
```

```
                        Qout = 1'b1;
```

```
                        if(Din == 1'b0)
```

```
                            NS = S4;
```

```
                        else
```

```
                            NS = S3;
```

```
                    end
```

```
                S5:
```

```
                    begin
```

```
                        Qout = 1'b0;
```

```
                        if(Din == 1'b0)
```

```
                            NS = S4;
```

```
                        else
```

```
                            NS = S0;
```

```
                    end
```

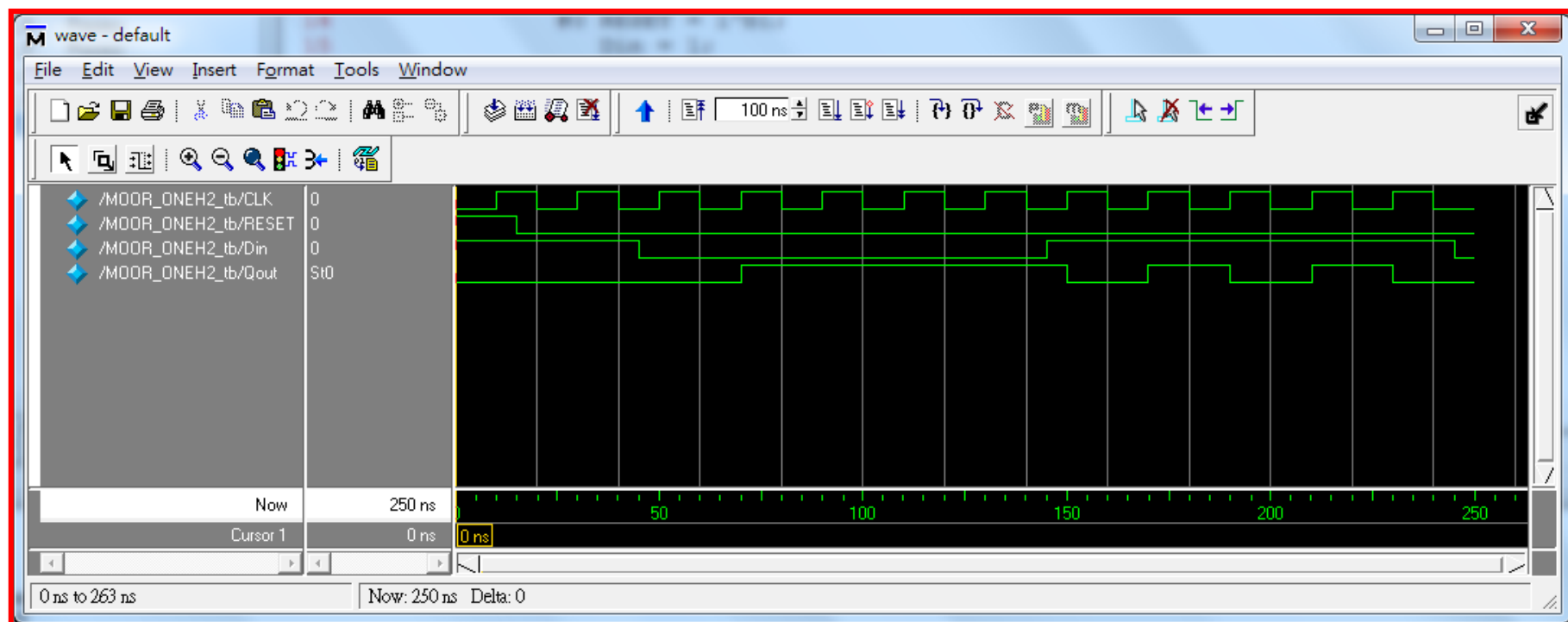
```
            endcase
```

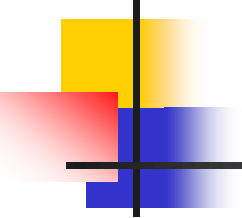
```
        end
```

```
    endmodule
```

Simulation Waveform

Code:10_002





範例練習

Mealy machine(10-003)

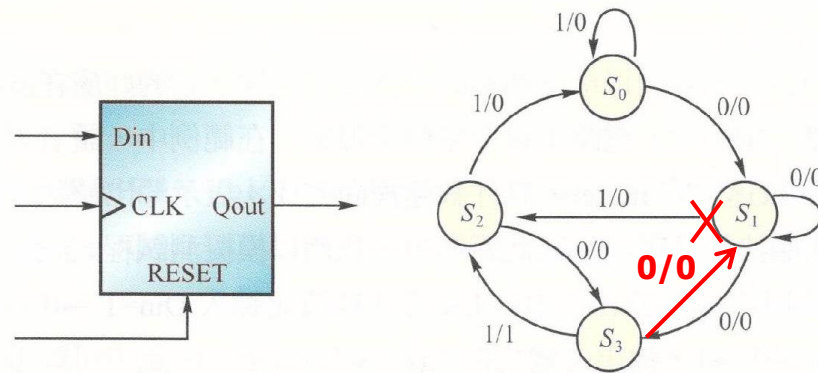
以格雷碼編碼法完成

由輸入信號中偵測出(0101)的電路

Mealy Machine with Gray Code

- 利用**格雷碼方式**來設計一個從輸入信號序列中偵測出(0101)的電路；即在狀態圖中，當偵測出(0101)信號時，則出現 $S_3 \Rightarrow S_2$ ，輸出為(1)。
- 動作範例：
- Initial status**
= S_0

Din	-	0	1	0	1	0	0	1	0	1	0
Q _{out}	0	0	0	0	1	0	0	0	0	1	0
Status	S_0	S_1	S_2	S_3	S_2	S_3	S_1	S_2	S_3	S_2	S_3



(a) 方塊圖

(b) 狀態圖

In/Out

目前狀態	下一個狀態	
	Din=0	Din=1
$S_0=00$	$S_1, 0$	$S_0, 0$
$S_1=01$	$S_1, 0$	$S_0, 0$
$S_0=11$	$S_3, 0$	$S_0, 0$
$S_3=10$	$S_1, 0$	$S_0, 1$

```
//Mealy machine with Gray encoded state machine
```

```
module MEALY_GRY2(CLK, Din, RESET, Qout);
```

```
    input CLK, RESET;
```

```
    input Din;
```

```
    output Qout;
```

```
    reg Qout;
```

```
    parameter[1:0]
```

```
        S0 = 2'b00,
```

```
        S1 = 2'b01,
```

```
        S2 = 2'b11,
```

```
        S3 = 2'b10;
```

```
    reg[1:0]CS;
```

```
    reg[1:0]NS;
```

```
    always@(posedge CLK or
            posedge RESET)
```

```
        begin
```

```
            if(RESET == 1'b1)
```

```
                CS = S0;
```

```
            else
```

```
                CS = NS;
```

```
        end
```

```
    always@(CS or Din)
```

```
        begin
```

```
            case(CS)
```

```
                S0:
```

```
                    begin
```

```
                        if(Din == 1'b0)
```

```
                            begin
```

```
                                NS = S1;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                        else
```

```
                            begin
```

```
                                NS = S0;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                    end
```

```
                S1:
```

```
                    begin
```

```
                        if(Din == 1'b0)
```

```
                            begin
```

```
                                NS = S1;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                        else
```

```
                            begin
```

```
                                NS = S2;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                    end
```

```
        S2:
```

```
            begin
```

```
                if(Din == 1'b0)
```

```
                    begin
```

```
                        NS = S3;
```

```
                        Qout = 1'b0;
```

```
                    end
```

```
                else
```

```
                    begin
```

```
                        NS = S0;
```

```
                        Qout = 1'b0;
```

```
                    end
```

```
            end
```

```
        S3:
```

```
            begin
```

```
                if(Din == 1'b0)
```

```
                    begin
```

```
                        NS = S1;
```

```
                        Qout = 1'b0;
```

```
                    end
```

```
                else
```

```
                    begin
```

```
                        NS = S2;
```

```
                        Qout = 1'b1;
```

```
                    end
```

```
            end
```

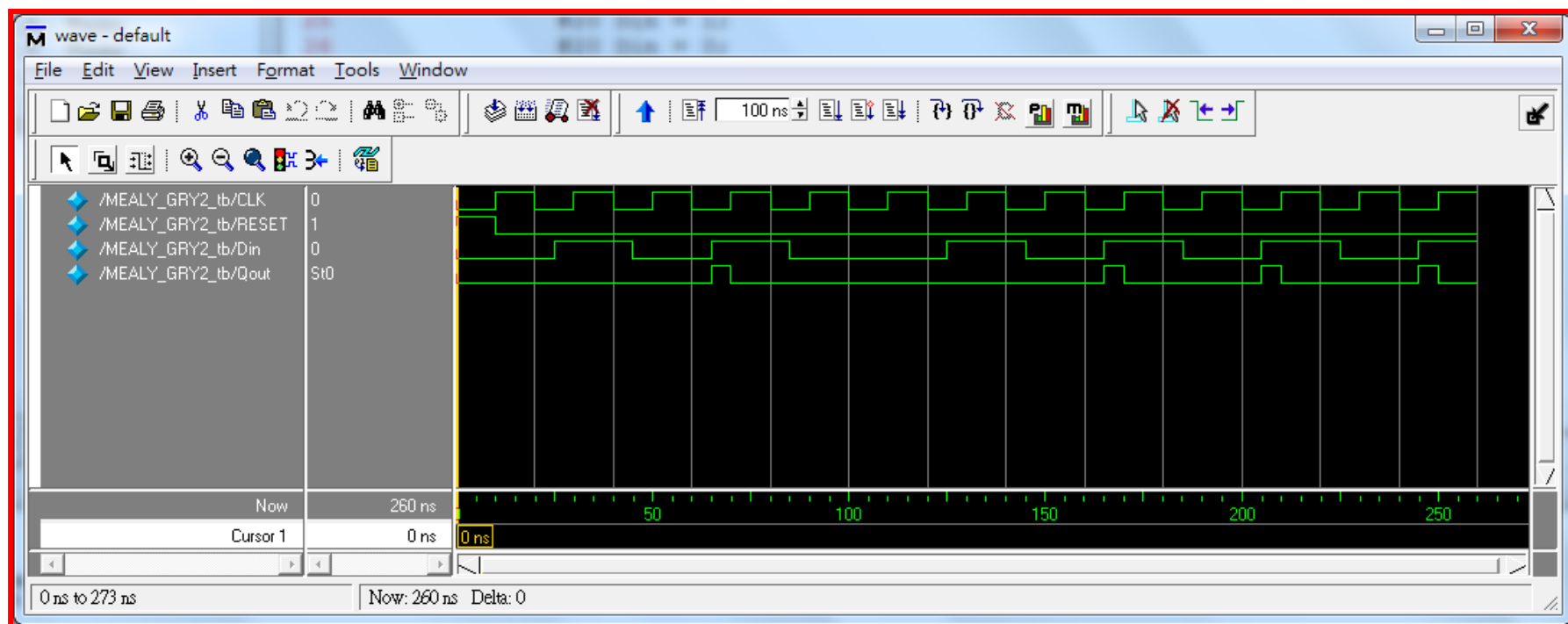
```
        endcase
```

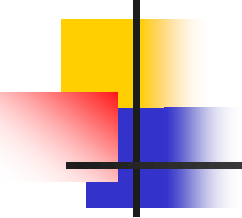
```
    end
```

```
endmodule
```

Simulation Waveform

Code:10_003





範例練習

Mealy machine(10-004)

以格雷碼編碼法完成由輸入信號中
偵測出(0101)或(1001)的電路

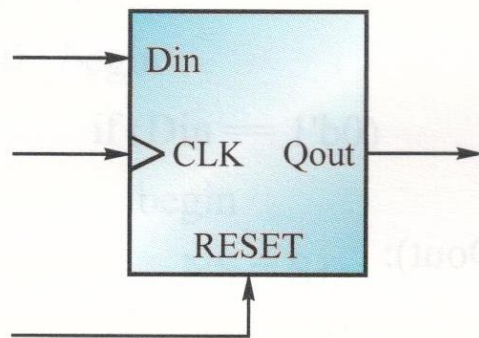
Mealy Machine with Gray Code

- 利用 **格雷碼方式** 來設計一個從輸入信號序列中，以連續四個位元為一個單位，偵測出(0101)或(1001)信號的電路；即只要輸入出現(0101)或(1001)時，輸出Qout為(1)。

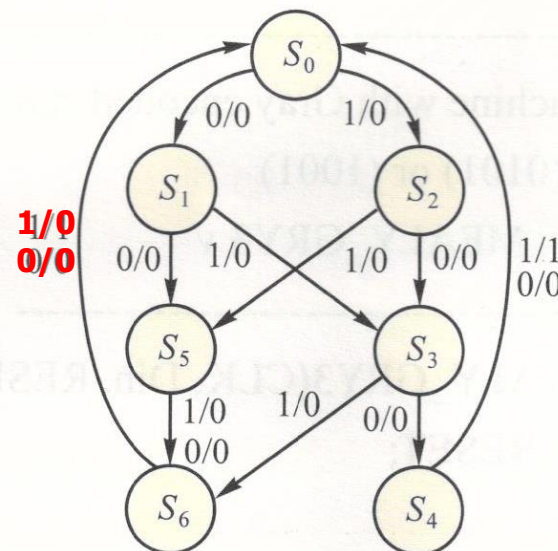
- 動作範例：

- Initial status**
= S_0

Din	-	0101	0110	001 1	1001
Q _{out}	0	0001	0000	000 0	0001
Status	S₀	S ₁ S ₃ S ₄ S ₀	S ₁ S ₃ S ₄ S ₀	S ₁ S ₅ S₆S₀	S ₁ S ₃ S ₄ S ₀



(a) 方塊圖



(b) 狀態圖

In/Put


```
//Mealy machine with Gray encoded state machine
```

```
module MEALY_GRY3(CLK, Din, RESET, Qout);
```

```
    input CLK, RESET;
```

```
    input Din;
```

```
    output Qout;
```

```
    reg Qout;
```

```
    parameter[2:0]
```

```
        S0 = 3'b000,
```

```
        S1 = 3'b001,
```

```
        S2 = 3'b011,
```

```
        S3 = 3'b010,
```

```
        S4 = 3'b110,
```

```
        S5 = 3'b111,
```

```
        S6 = 3'b101;
```

```
    reg[2:0]CS;
```

```
    reg[2:0]NS;
```

```
    always@(posedge CLK or posedge RESET)
```

```
        begin
```

```
            if(RESET == 1'b1)
```

```
                CS = S0;
```

```
            else
```

```
                CS = NS;
```

```
        end
```

(1)

```
    always@(CS or Din)
```

```
        begin
```

```
            case(CS)
```

```
                S0:
```

```
                    begin
```

```
                        if(Din == 1'b0)
```

```
                            begin
```

```
                                NS = S1;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                        else
```

```
                            begin
```

```
                                NS = S2;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                    end
```

```
                S1:
```

```
                    begin
```

```
                        if(Din == 1'b0)
```

```
                            begin
```

```
                                NS = S5;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                        else
```

```
                            begin
```

```
                                NS = S3;
```

```
                                Qout = 1'b0;
```

```
                            end
```

```
                    end
```

(2)

```

S2:
begin
  if(Din == 1'b0)
    begin
      NS = S3;
      Qout = 1'b0;
    end
  else
    begin
      NS = S5;
      Qout = 1'b0;
    end
  end

S3:
begin
  if(Din == 1'b0)
    begin
      NS = S4;
      Qout = 1'b0;
    end
  else
    begin
      NS = S6;
      Qout = 1'b0;
    end
  end

S4:
begin
  if(Din == 1'b0)
    begin
      NS = S0;
      Qout = 1'b0;
    end
  else
    begin
      NS = S0;
      Qout = 1'b1;
    end
  end
end

```

(3)

```

S5:
begin
  if(Din == 1'b0)
    begin
      NS = S6;
      Qout = 1'b0;
    end
  else
    begin
      NS = S6;
      Qout = 1'b0;
    end
  end

S6:
begin
  if(Din == 1'b0)
    begin
      NS = S0;
      Qout = 1'b0;
    end
  else
    begin
      NS = S0;
      Qout = 1'b0;
    end
  end
endcase
end

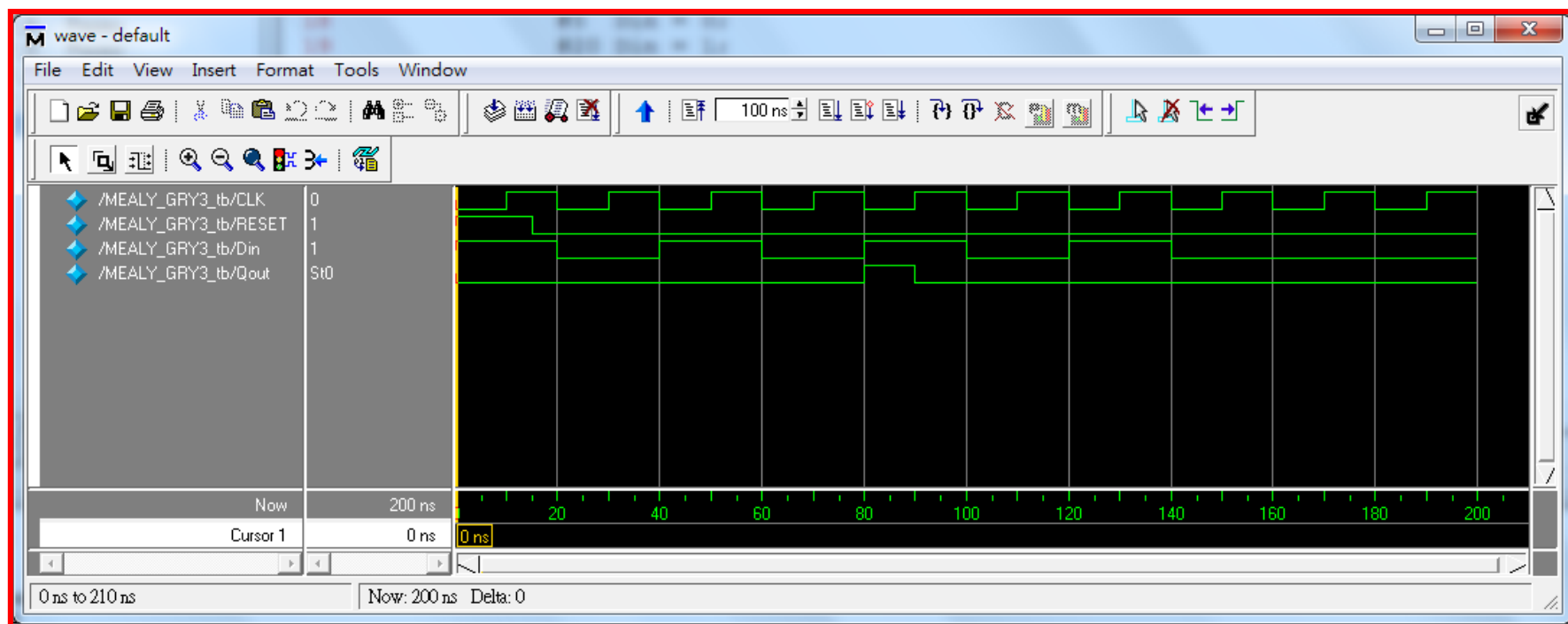
endmodule

```

(4)

Simulation Waveform

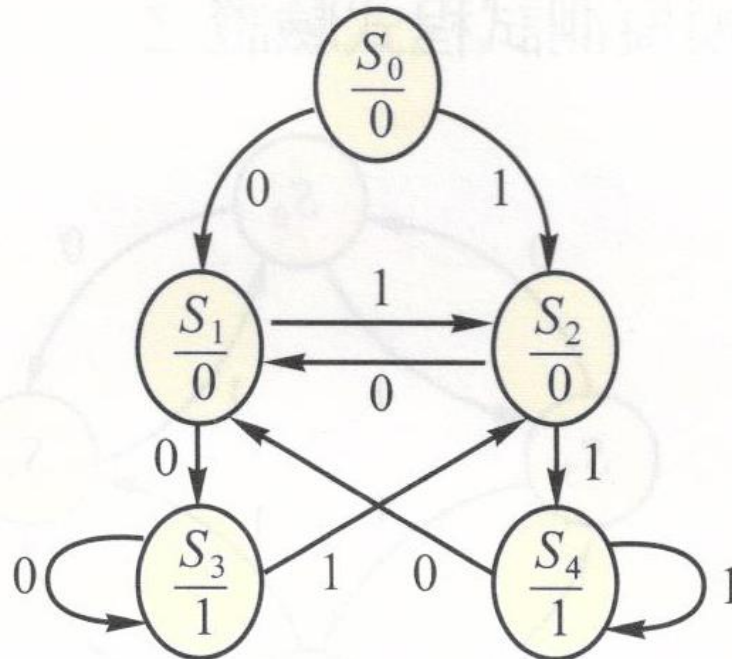
Code:10_004



範例練習 (Homework)

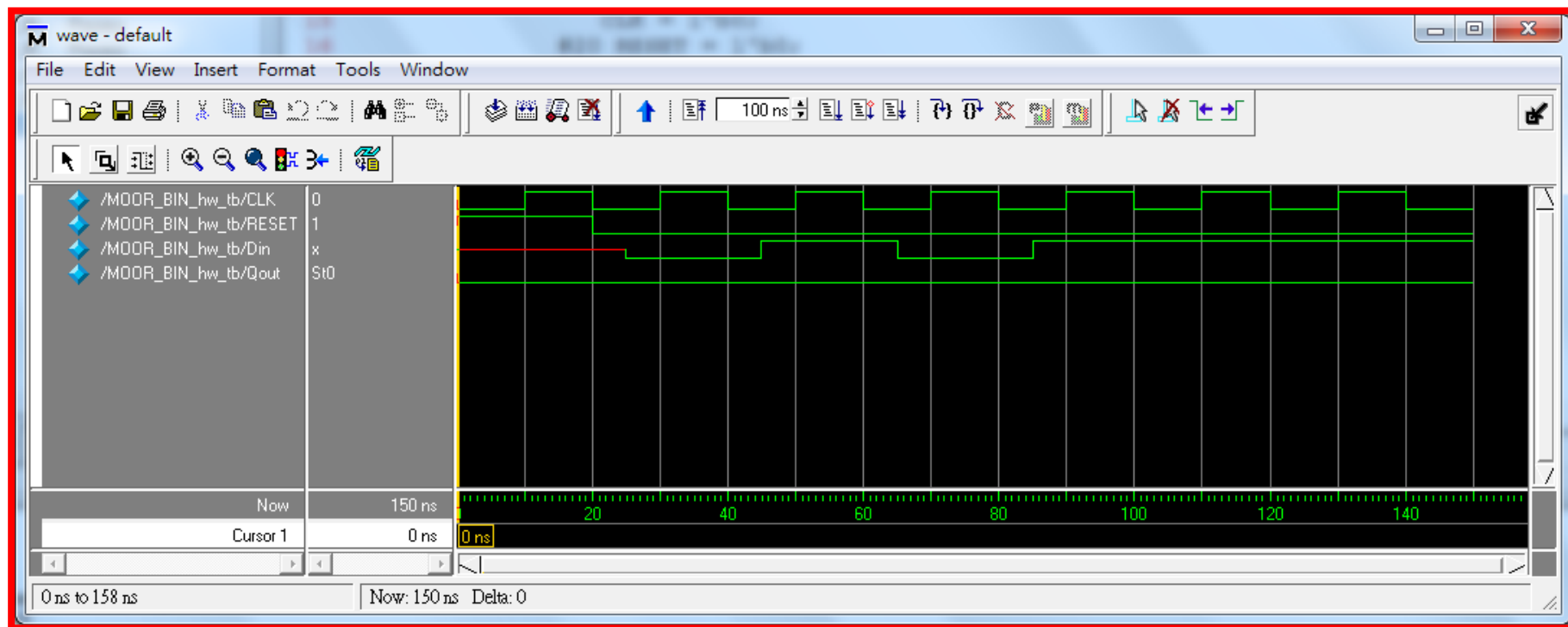
Moore machine(10-005)

Din	-	0	1	0	1	0	0	1	0	1	0
Q_{out}	0										
Status	S_0										



Simulation Waveform

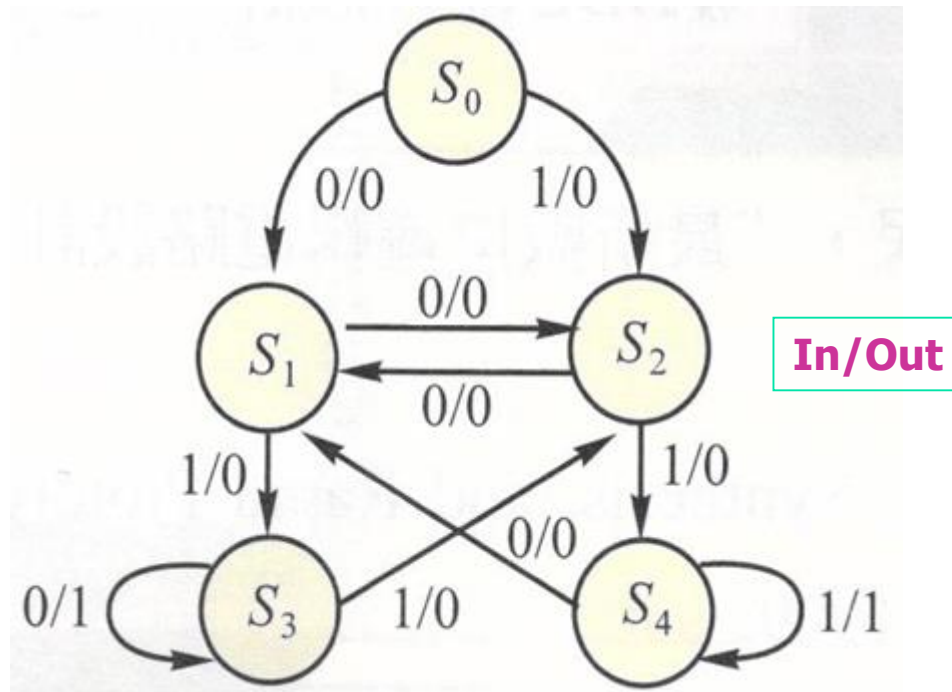
Code:10_005



範例練習

Mealy machine(10-006)

Din	-	0	1	0	1	0	0	1	0	1	0
Q _{out}	0										
Status	S ₀										



Simulation Waveform

Code:10_006

